



Chapter One

Introduction to Distributed Systems



Contents

- ❖ Introduction
 - ❖ Definition
 - ❖ Characteristics of Distributed Systems
 - ❖ Goals of a Distributed System
 - ❖ Types of Distributed Systems
-



Introduction

- ❖ Computer systems are undergoing a revolution.
 - ❖ From 1945, when the modern computer era began, until about 1985, computers were large and expensive. Even minicomputers cost at least tens of thousands of dollars each.
 - ❖ Starting around the mid-1980s, however, two advances in technology began to change that situation. The first was the development of powerful micro processors. Initially, these were 8-bit machines, but soon 16-, 32-, and 64-bit CPUs became common.
-



Introduction

- ❖ The second development was the invention of **high-speed computer networks**.
 - ❖ **Local-area networks** or LANs allow hundreds of machines within a building to be connected in such a way that small amounts of information can be transferred between machines in a few microseconds.
 - ❖ Larger amounts of data can be moved between machines at rates of **100 million to 10 billion bits/sec**.
 - ❖ **Wide-area networks or WANs** allow million of machines all over the earth to be connected at speeds varying from 64 Kbps (kilobits per second) to gigabits per second.
-



Introduction

- ❖ The result of these technologies is that it is now not only **feasible**, but easy, to put together computing systems composed of large numbers of computers connected by **a high-speed network**.
- ❖ They are usually called computer networks or distributed systems.
- ❖ This is different from older centralized systems, where all processing was done by a single computer. That single machine handled everything using its own hardware, peripherals like printers or storage, and sometimes a few remote terminals that only provided input and output but did not do real processing.



Definition of Distributed System

- ❖ A distributed system is a collection of independent computers that appears to its **users** as a single coherent system.
- ❖ This definition has two aspects: The first one deals with hardware: the machines are autonomous. The second one deals with software: the users think of the system as a single computer.
- ❖ They help in sharing different **resources** and **capabilities** to provide users with a single and integrated coherent network.
- ❖ A distributed system is one in which **components** located at networked computers, communicate and **coordinate** their actions only by-passing **messages**.



Definition of Distributed System

- ❖ **Ideal:** to present a single-system image: the distributed system “looks like” a single computer rather than a collection of separate computers.
- ❖ A distributed system contains **multiple nodes** that are physically separate but linked together using the network.
- ❖ All the nodes in this system communicate with each other and handle processes in tandem(together at the same time while working in coordination). Each of these nodes contains a small part of the distributed operating system software.
- ❖ The opposite of a distributed system is a **centralized system where** all of the components of a computing system reside in one machine.

Definition of Distributed System

- ❖ To support heterogeneous computers and networks and to provide a single-system view, a distributed system is often organized by means of a layer of software called **middleware** that extends over multiple machines.
- ❖ **Middleware** is the software layer that sits **between the operating system and the applications**. It runs across all machines in the system and handles: Communication between nodes ,Data consistency, Resource management and Coordination of processes

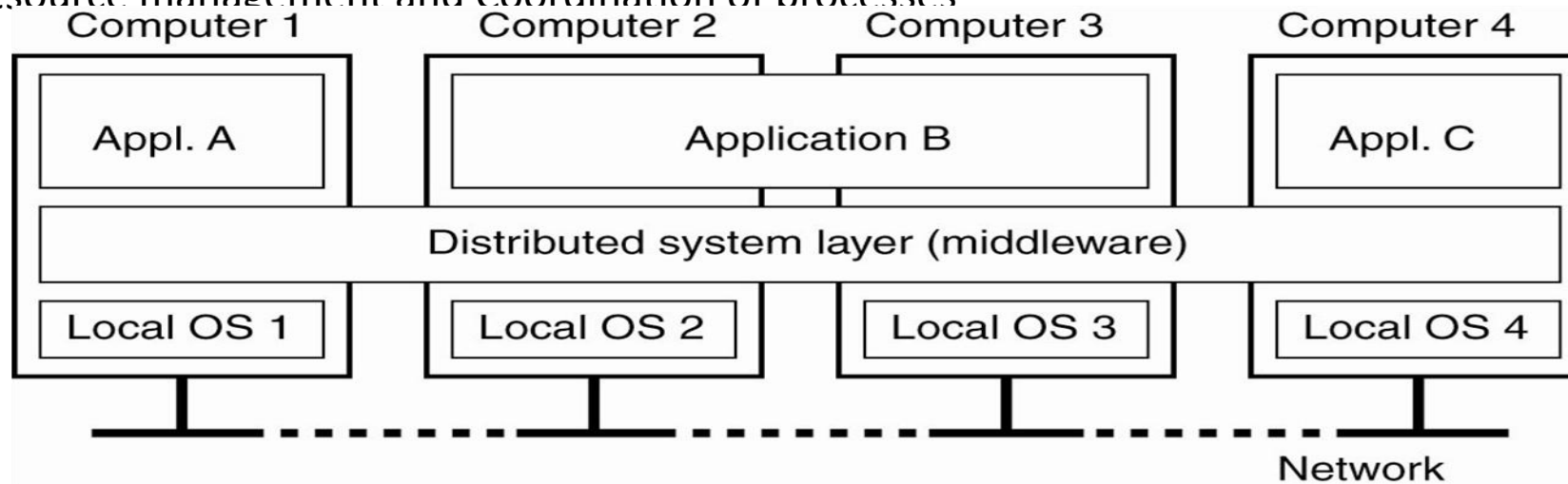


Figure: a distributed system organized as middleware; note that the middleware layer extends over multiple machines, and offers each application the same interface



Role of Middleware

- ❖ **Middleware** is software that functions as intermediate layer between **applications** and **OS** or between client and server.
 - ❖ Early distributed systems tried to make multiple independent machines appear as a single unified system. This is often called transparency, and middleware provides this illusion by handling communication, synchronization, and resource sharing behind the scenes.
 - ❖ Today:- clustering software allows **independent** computers to work together closely.
-



Middleware Examples

- ❖ **CORBA** (Common Object Request Broker Architecture)
 - ❖ **DCOM** (Distributed Component Object Management)- Bing replaced by .NET
 - ❖ **Sun's ONC RPC** (Open Network Computing Remote Procedure Call)
 - ❖ **RMI** (Remote Method Innovation)
 - ❖ **SOAP** (Simple Object Access Protocol)
-



Characteristics of Distributed Systems

- ❖ **Multiple Autonomous (independent) Components:** The system consists of multiple autonomous components (nodes or processes) that are physically separate and can operate independently. Each node has its own CPU and memory and can fail separately.
- ❖ **Resource Sharing:** This is a primary motivation. Components can share hardware software (e.g., a distributed database), and data (e.g., shared files). Users access these remote resources as if they were local.
- ❖ **Concurrent Processing:** Multiple components operate simultaneously. Programs run concurrently and often need to coordinate their actions, for example, to access shared resources.



Characteristics of Distributed Systems

- ❖ **Message-Passing Communication:** The separate components coordinate their actions and exchange information solely by-passing messages over a network (e.g., HTTP, RPC, queues).
- ❖ **Lack of a Global Clock / Asynchronous Nature:** It is extremely difficult to have a single, global notion of time that is perfectly synchronized across all nodes. This makes reasoning about the order of events in a distributed system a complex challenge.
- ❖ **Heterogeneity:** The system can be built from a variety of different hardware, networks, operating systems, programming languages, and implementations by different vendors. The system must be designed to handle this diversity.



Advantages of distributed systems over centralized systems.

ITEM	Description
Economics	Microprocessors offer a better price/performance than mainframes
Speed	A distributed system may have more total computing power than a mainframe
Inherent distribution	In a distributed system, the components (computers/nodes) are physically separate and located in different places by design(plan)
Reliability	If one machine crashes, the system as a whole can still survive
Incremental growth	Computing power can be added in small increments



Advantages of distributed systems over isolated (personal) computers

Item	Description
Data Sharing	Allow many users access to a common data base
Device sharing	Allow many users to share expensive peripherals like color printers
Communication	Make human-to-human communication easier, for example, by electronic mail.
Flexibility	Spread the workload over the available machines in the most cost-effective way



Disadvantages of distributed systems

Item	Description
Software	Little software exists at present for distributed systems
Networking	The network can saturate or cause other problems
Security	Easy access also applies to secret data



Goals of a Distributed

- ❖ Resource Accessibility
- ❖ Distribution Transparency
- ❖ Openness
- ❖ Scalability



Goal 1- Resource Accessibility

- ❖ Support user to **access** remote resources (data files, web pages, CPU cycles) and the **fair** sharing of the resources.
 - ❖ Economics of the sharing **expensive** resources.
 - ❖ Performance enhancement – due to multiple processors, ease of collaboration and information exchange- access to remote services.
 - **Groupware**: is tools to support collaboration.
 - ❖ Resource sharing introduces **security** problems.
-



Goal – 2 Distribution

Transparency

- ❖ An important goal of a distributed system is to hide the fact that its processes and resources are physically distributed across multiple computers.
- ❖ Make the system more user friendly.
- ❖ A distributed system that appears to its **users** and **application** to be a **single** computer system is said to be *transparent*.
 - Users and apps should be able to access remote resources in the same way they access **local** resources.



Types of Transparency

Transparency	Description
Access	Hide differences in data representation and how a resource is accessed
Location	Hide where a resource is located
Migration	Hide that a resource may move to another location
Relocation	Hide that a resource may be moved to another location while in use
Replication	Hide that a resource is replicated
Concurrency	Hide that a resource may be shared by several competitive users
Failure	Hide the failure and recovery of a resource



Goal – 3 Openness

- ❖ An open distributed system is a system that offers services according to standard rules that describe the **syntax and semantics** of those services. In other words, the interfaces to the system are clearly specified and freely available.
- ❖ For example, in computer networks, standard rules govern the **format, contents, and meaning of messages sent and received**. Such rules are formalized in protocols.
- ❖ In distributed systems, services are generally specified through interfaces, which are often described in an **Interface Definition Language (IDL)**.
- ❖ **Example:**
 - ❖ In a network, sending an email requires following **SMTP protocol**. Any email client that follows SMTP can send mail, no matter which software or vendor it uses.
 - ❖ In distributed systems, middleware might define a service for “get customer data.” If it’s **open**, any client that follows the interface rules can use it.



Goal – 3 Openness Con'd

- ❖ Interface definitions written in an IDL nearly always capture only the syntax of services.
- ❖ **Interface Definition Languages (IDL):** used to describe the interfaces between **software** components, usually in a distributed system.
 - Definitions are language and machine independent
 - Support communication between systems using different **OS/ programming** languages; e.g. a C++ program running on Windows communicates with a java program on UNIX.



Open Systems Support...

- ❖ **Interoperability:** the ability of **two** different systems or applications to work **together**.
 - A process that **needs** a service should **be able to talk** to any process that **provides** the service.
 - Multiple implementations of the same service may be provided, as long as the interface is maintained.
- ❖ **Portability:** an application designed to run on one distributed system can run **on another** systems which implements the same interface.
- ❖ **Extensibility:** Easy to add **new** components, features.



Goal – 4 Scalability

- ❖ Scalability of a system can be measured along at least three different dimensions.
- ❖ First, a system can be scalable with respect to its **size**, meaning that we can easily add more users and resources to the system.
- ❖ Second, **a geographically** scalable system is one in which the users and resources may lie far apart.
- ❖ Third, a system can **be administratively** scalable, meaning that it can still be easy to manage even if it spans many independent administrative organizations.



Size Scalability

- ❖ Scalability is **negatively** affected when the system is based on;
 - Centralized server: one for all users.
 - Centralized data: a single database for all users.
 - Centralized algorithms: one site collects all information, processes it, distributes the results to all sites.



Geographic Scalability

- ❖ Early distributed systems ran on LANs, relied on **Synchronous communication**(node sends a message and **waits for a reply** before continuing.)
 - May be too slow for WAN.
 - Wide-area communication is unreliable.
 - Unpredictable time delays may even affect correctness.
- ❖ LAN communication is based on Broadcast.
- ❖ Centralized components + wide area communication = wastage of network bandwidth.



Administrative Scalability

- ❖ It describes **how easily multiple independent authorities can manage and operate parts of the same distributed system** without causing conflicts or excessive complexity.
- ❖ **Trust often stops at administrative boundaries**
 - In a distributed system, different parts may belong to different organizations or authorities.
 - Each domain usually trusts its own users and resources, but does not automatically trust others.
 - So when components communicate across domains, the system must use: authentication, authorization and secure communication.



Scaling Techniques

- ❖ Scalability affects **performance** more than anything else.
- ❖ **Three** techniques to improve scalability:
 - Hiding communication latencies/ delay.
 - Distribution
 - Replication



1. Hiding Communication Delays

- ❖ Structure applications to use asynchronous communication (no blocking for replies)
 - While waiting for one answer, do something else; e.g., create one thread to wait for the reply and let other threads continue to process or schedule another task.
- ❖ Move part of the work to the client so it can process locally, and send data in smaller parts or as a complete batch to reduce waiting time.

1. Hiding Communication

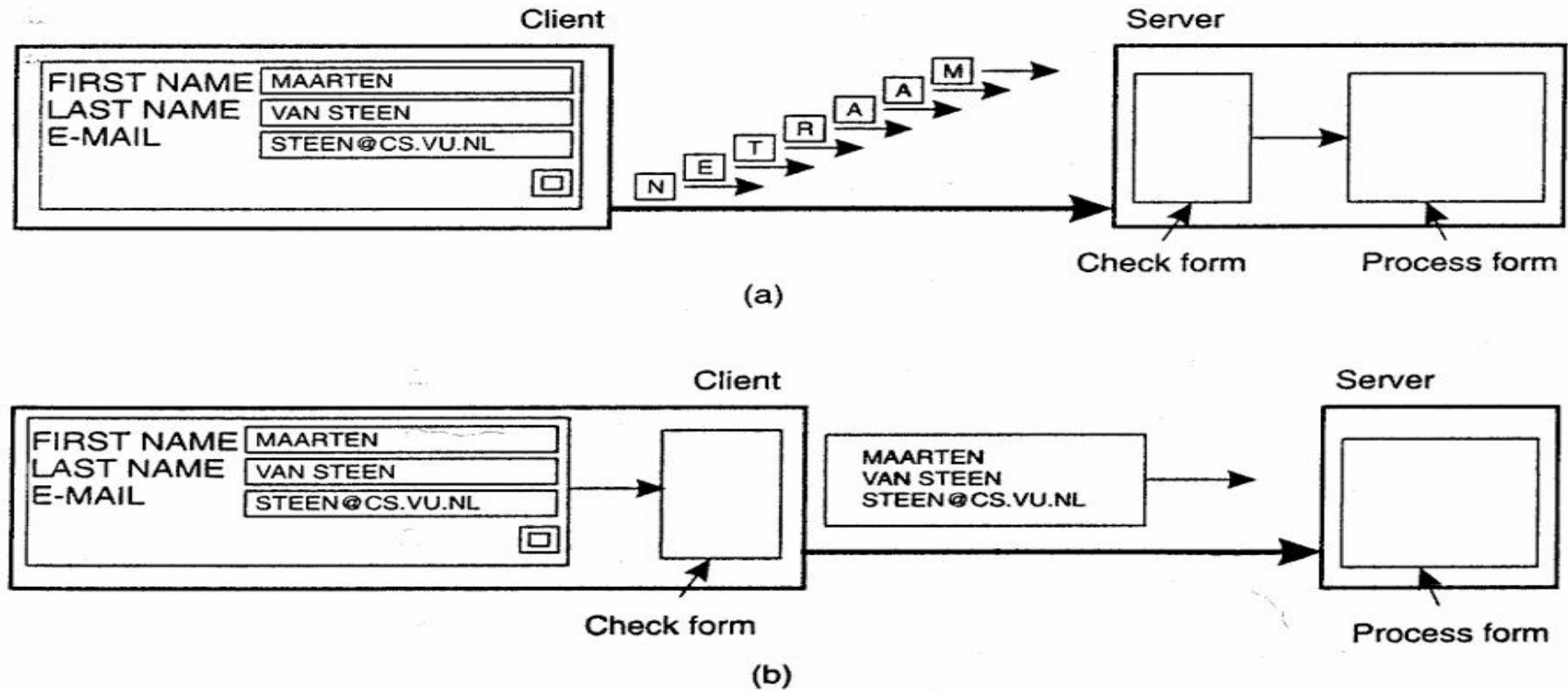


Figure 1-4. The difference between letting (a) a server or (b) a client check forms as they are being filled.



2. Distribution

- ❖ Instead of one centralized service, divide into parts and distribute geographically.
- ❖ Each part handles one aspect of the job
- ❖ Download part of the computation to the requesting platform to **speed up** processing.
 - Example: DNS namespace is organized as a tree of domains(eg .org .com ,.edu); each domain is divided into zones; names in each zone are handled by a different name server.
 - WWW consists of many (millions?) of servers.

Scaling techniques

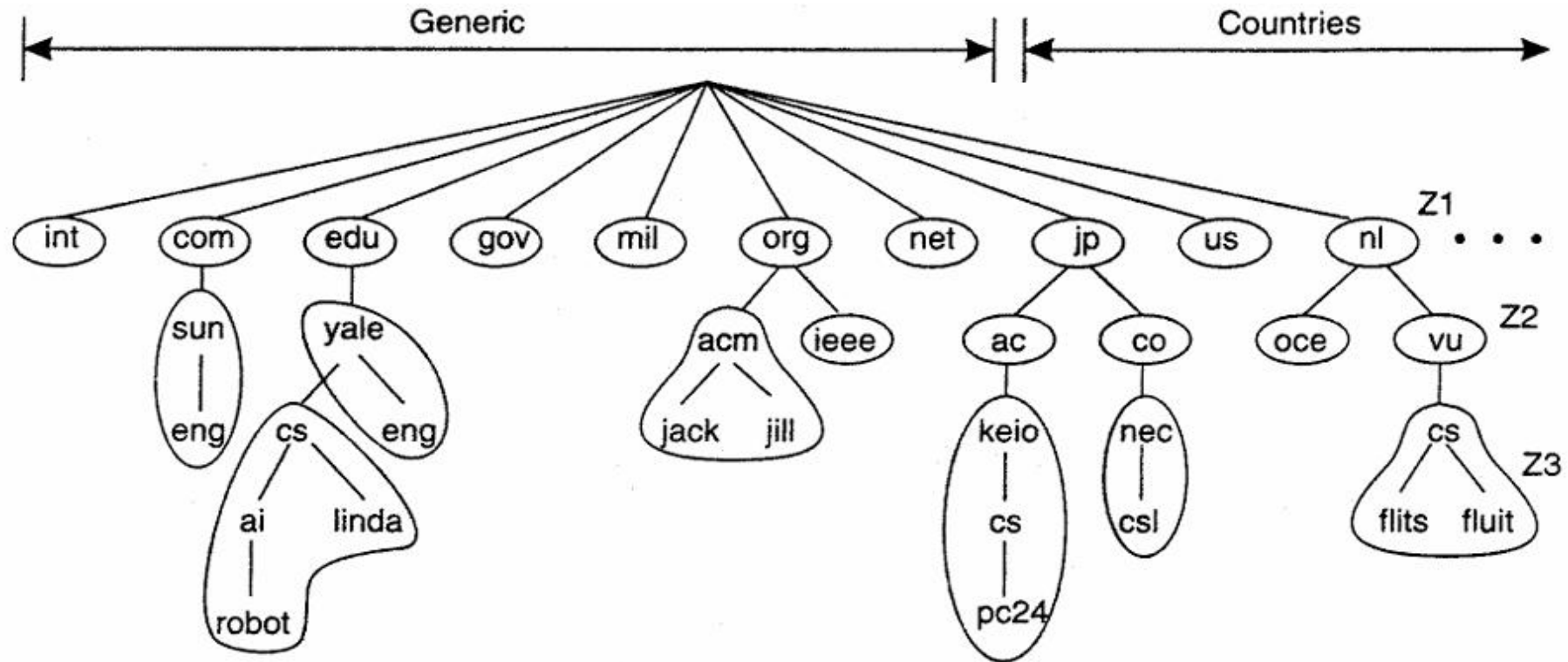


Figure 1-5. An example of dividing the DNS name space into zones.



3. Replication

- ❖ **Replication:** multiple identical copies of something
 - Replicated objects may also be distributed, but aren't necessarily.
- ❖ Replication;
 - Increases availability
 - Improves performance through load balancing
 - May avoid latency by improving proximity of resource



Types of Distributed Systems

- ❖ Distributed Computing Systems (1)
 - Clusters
 - Grids
 - Clouds
- ❖ Distributed Information Systems (2)
 - Transaction Processing Systems
 - Enterprise Application Integration
- ❖ Distributed Embedded Systems (3)
 - Home systems
 - Health case systems
 - Sensor networks



1. Distributed Computing Systems

A. Cluster Computing

- ❖ A collection of similar processors (PCs, workstations) running the **same** OS, connected by high-speed LAN.
- ❖ Parallel computing capabilities using **inexpensive** PC hardware.
- ❖ Clustering is primarily used for **high** availability and fault tolerance
- ❖ If one node **fails**, another node in the cluster can take over, ensuring continuous operation of the system.



Cluster Types and Uses

- ❖ High Performance Clusters (HPC)
 - Run large parallel programs.
 - Scientific, military, engineering apps; e.g., weather modeling.
 - **Key feature:** Focuses on **speed and performance** using many nodes working together.
- ❖ Load balancing clusters (LBC)
 - Distribute incoming requests among multiple servers to **balance the load**.
 - Server farms (e.g., at banks or popular web site)
 - **Key feature:** Improves **throughput and response time** by preventing any single server from being overloaded.
- ❖ High Availability Clusters (HAC)
 - Provide redundancy – back up systems.
 - **Where used:** Critical systems that **cannot afford downtime**.
Example Airline reservation systems
 - More fault tolerant.



Distributed Computing Systems

B. Grids

- ❖ Similar to clusters but processors are more **loosely** coupled, tend to be **heterogeneous**, and are not all in a central location.
- ❖ Can handle workloads similar to those on supercomputers, but grid computers connect over a network (Internet) and supercomputer's CPUs connect to a high-speed internal bus/network.
- ❖ Problems are **broken** up into parts and distributed across multiple computers in the **grid-less communication** between parts than in clusters.



Distributed Computing Systems

A proposed Architecture for Grid Systems

- ❖ **Fabric layer:** interfaces to local resources at a specific site.
 - ❖ **What it is:** The **physical or logical resources** you want to use. e.g. CPUs, storage disks, sensors, databases, clusters of computers.
- ❖ **Connectivity layer:** protocols to support usage of multiple resources for a single applications; e.g., access a remote resources or transfer data between resources; and protocols to provide security.
- ❖ **Resource layer:** manages a single resource, using functions supplied by the connectivity layer. Examples: Job scheduler on a supercomputer
- ❖ **Collective layer: usually parts of middleware,** manages groups of resources; coordinates multiple nodes. Examples: Resource discovery tools (find available computers)
- ❖ **Applications:** Programs that **actually perform tasks using the grid resources. Examples: Weather simulation programs**
- ❖ The collective, connectivity and resource layers together form the middleware layer for a grid.



Distributed Computing Systems

A proposed Architecture for Grid Systems

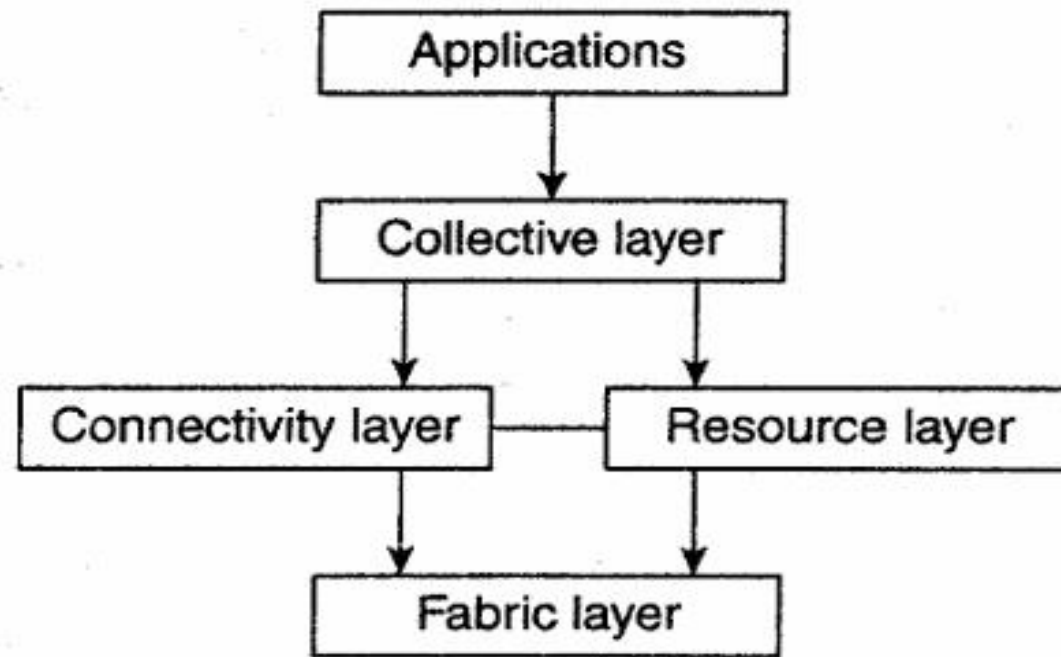


Figure: A layered architecture for grid computing systems.



Distributed Computing Systems

C. Cloud Computing

- ❖ Provides scalable services as a utility over the internet.
 - ❖ Often built on a computer grid.
 - ❖ Users buy services from the cloud
 - Grid users may develop and run their own software.
 - ❖ Cluster/ grid/ cloud distinctions blur at the edges!. i.e the boundaries between clusters, grids, and clouds aren't always clear-cut. In practice, systems often combine features from more than one model
-



2. Distributed Information Systems

- ❖ Business-oriented
- ❖ Systems to make a number of separate network applications interoperable and build “enterprise-wide information systems”
- ❖ Two types discussed here:
 - Transaction Processing systems
 - Enterprise application integration (EAI)



Distributed Information Systems

A. Transaction Processing Systems

- ❖ Provide a **highly** structured client-server approach for database applications.
 - ❖ Transactions are the communication model.
 - ❖ Obey the ACID properties:
 - Atomic: all or nothing
 - Consistent: invariants / constants are preserved.
 - Isolated (Serializable)
 - Durable: committed operations can't undone
-



Distributed Information Systems

A. Transaction Processing Systems

Primitive	Description
BEGIN_TRANSACTION	Mark the start of a transaction
END_TRANSACTION	Terminate the transaction and try to commit
ABORT_TRANSACTION	Kill the transaction and restore the old values
READ	Read data from a file, a table, or otherwise
WRITE	Write data to a file, a table, or otherwise

Figure: Example primitives for transactions.



Distributed Information Systems

Transactions

- ❖ Transaction processing may be **centralized** (traditional client/ server systems) or **distributed**.
- ❖ A distributed database is one in which the data storage is distributed-connected to separate processors.



Distributed Information Systems

Nested Transactions

- ❖ A nested transaction is a transaction within another transactions (a sub-transaction)
 - Example: a transaction may ask for two things (e.g., airline reservation info + hotel info) which would spawn two nested transactions.
- ❖ Primary transaction waits for the results.
 - While children are active, parent may only abort, commit, or spawn other children



Distributed Information Systems

Transaction Processing Systems

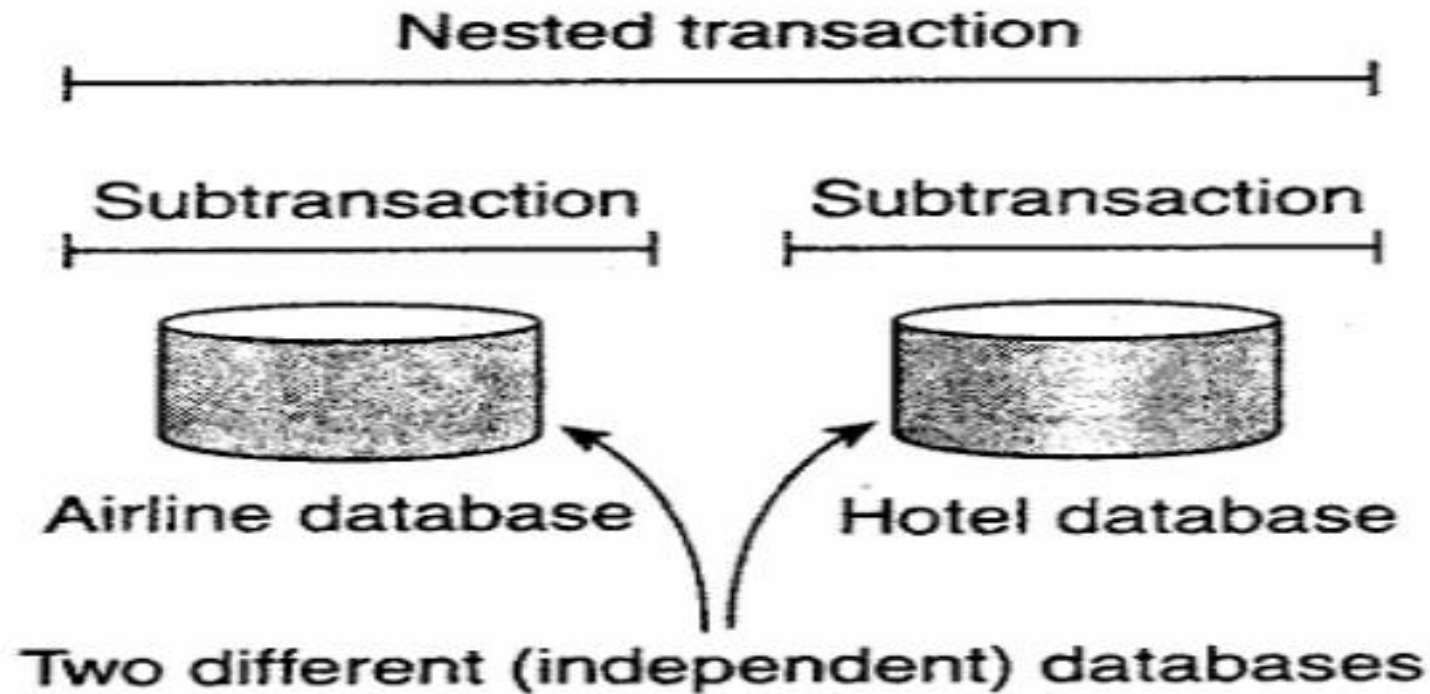


Figure: a nested transaction.



Distributed Information Systems

Enterprise Application Integration

- ❖ Less structured than transaction-based systems
 - ❖ EA components communicate directly.
 - Enterprise applications are things like HR data, inventory programs,....
 - May use different Oss, different DBs but need to interoperate sometimes.
 - ❖ Communication mechanisms to support this include CORBA, Remote Procedure Call (RPC) and Remote Method Invocation (RMI)
-



Distributed Information Systems

Enterprise Application

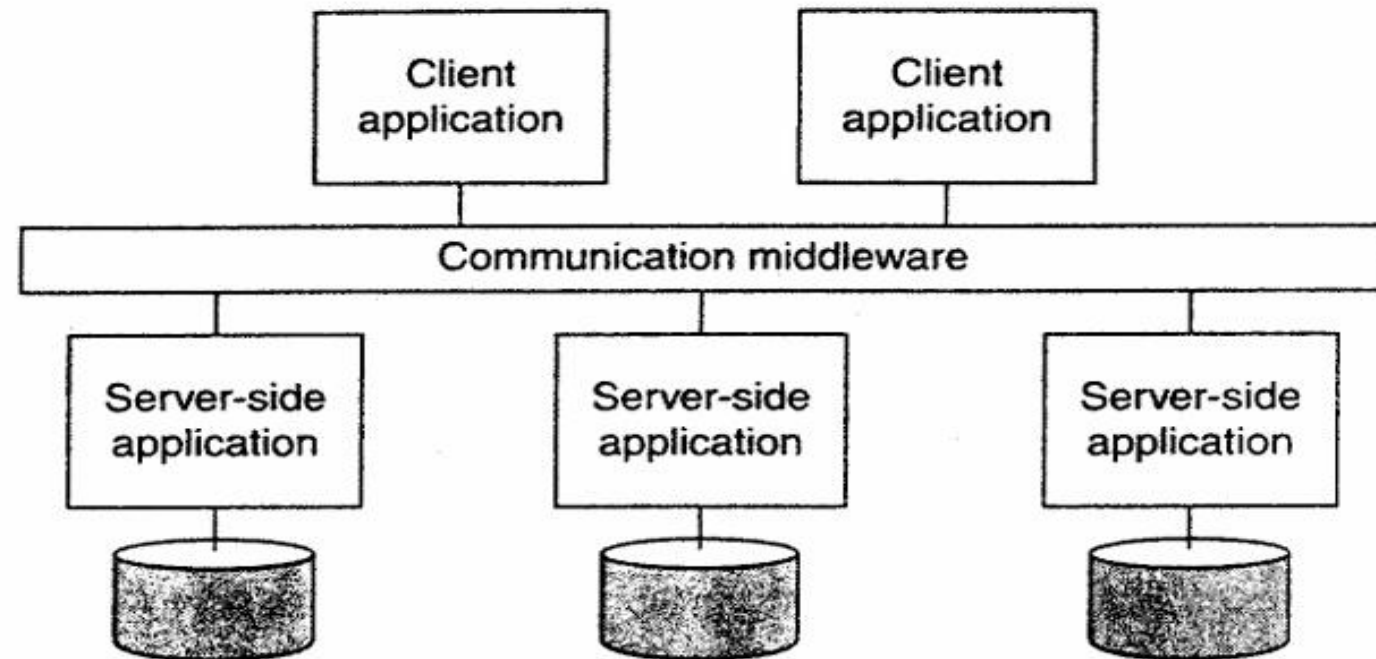


Figure: Middleware as a communication facilitator in enterprise application integration.



Distributed Pervasive/Embedded Systems

- ❖ Provide smart services using many small devices
- ❖ Examples:
 - ❖ **Home systems:** Smart homes, IoT devices
 - ❖ **Healthcare systems:** Patient monitoring, wearable sensors
 - ❖ **Sensor networks:** Environmental or industrial monitoring
- ❖ This type of system is likely to incorporate small, battery-powered,



3. Distributed Pervasive/Embedded Systems

A. Home System

- ❖ Built around one or more PCs, but can also include other electronic devices:
 - Automatic control of lighting, sprinkler systems, alarm systems, etc.
 - Network enabled appliances
 - PDAs(Personal Digital Assistant, etc.

3. Distributed Pervasive/Embedded Systems

B. Electronic Health Care Systems

- ❖ Personal health care systems are often equipped with various sensors organized in a (preferably wireless) body-area network (BAN).

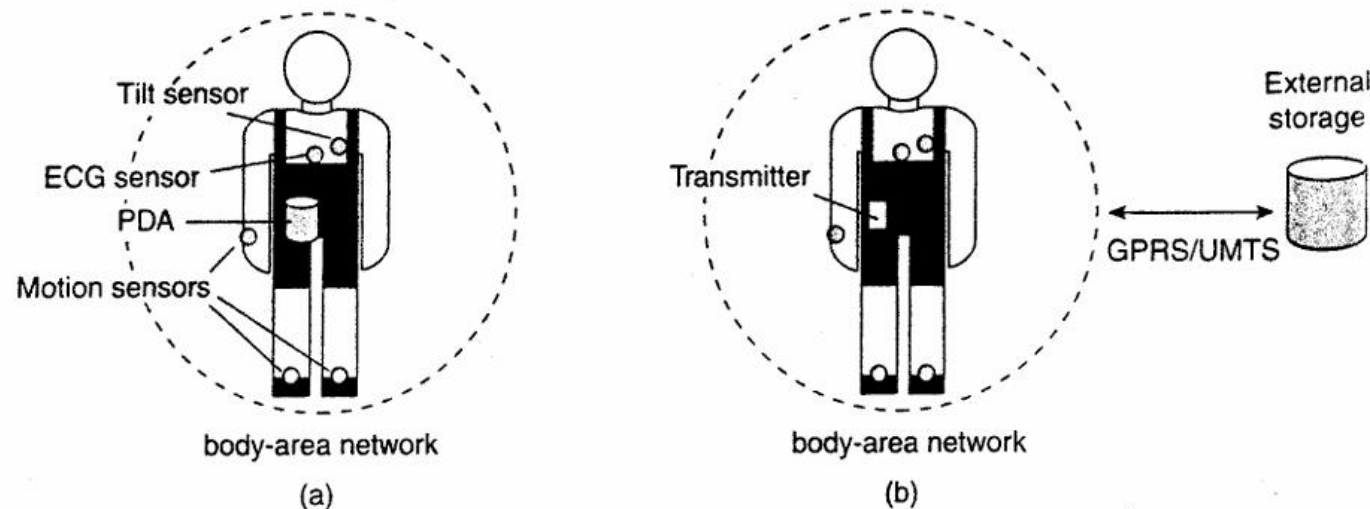


Figure: Monitoring a person in a pervasive electronic health care system, using (a) a local hub or (b) a continuous wireless connection.



3. Distributed Pervasive/Embedded Systems

C. Sensor Networks

- ❖ A collection of geographically distributed nodes consisting of a common device, a power source, some kind of sensor, a small processor...
- ❖ **Purpose:** to collectively monitor sensory data (temperature, sound, moisture etc.,) and transmit the data to a base station.
- ❖ “smart environment”- the nodes may do some rudimentary processing of the data in addition to their communication responsibilities.

3. Distributed Pervasive/Embedded

C. Sensor Network

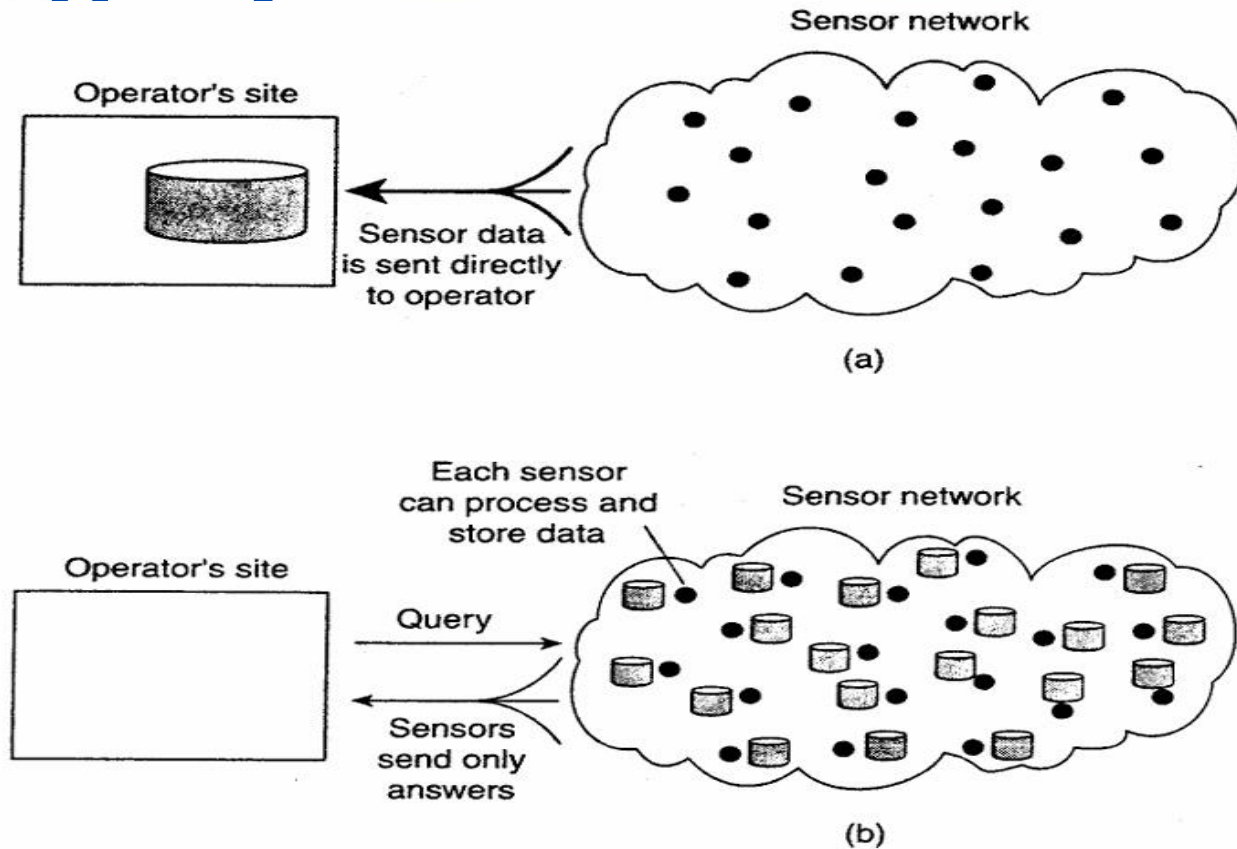


Figure: Organizing a sensor network database, while storing and processing data (a) only at the operator's site or (b) only at the sensors.



**Thank
k You**
